Problem Context

A perceptron is a machine learning model that tries to find a straight line to separate two classes of data. Given a dataset of points with coordinates (x1, x2) and binary labels (0 or 1), the goal is to learn weights w1, w2 and a bias b such that the line W1x1 + W2x2 + b = 0 correctly separates the two classes. This assignment implements and compares two approaches to learning those weights. A heuristic approach and a gradient descent approach.

Solutions: Part 1 - Heuristic Perceptron

The heuristic perceptron uses a step function to classify each point as 0 or 1. It only updates weights when a point is misclassified. If the prediction was 0 but should be 1, weights are increased. If the prediction was 1 but should be 0, weights are decreased:

```
def step(x):
    return 1 if x >= 0 else 0

y_hat = step(w1 * x1_i + w2 * x2_i + b)

if y_hat != y_i:
    if y_hat == 0:
        b  = b  + learning_rate
        w1 = w1 + learning_rate * x1_i
        w2 = w2 + learning_rate * x2_i
    else:
        b  = b  - learning_rate
        w1 = w1 - learning_rate * x1_i
        w2 = w2 - learning_rate * x2_i
```

Solutions: Part 2 - Gradient Descent Perceptron

The gradient descent perceptron replaces the step function with a sigmoid function, giving a continuous probability output between 0 and 1. Unlike Part 1 it updates weights for every point every epoch, not just misclassified ones. The update size depends on how wrong the prediction was through the error y - ŷ:

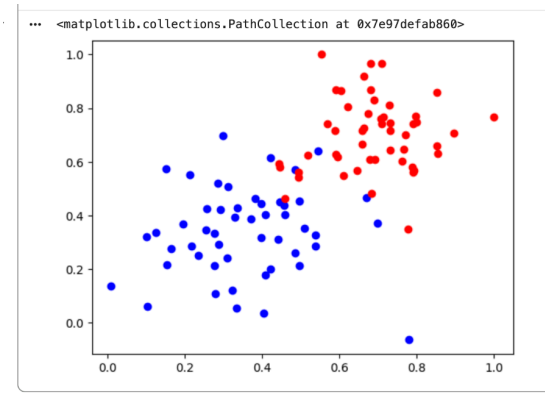
```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

y_hat = sigmoid(w1 * x1_i + w2 * x2_i + b)
error = y_i - y_hat

b  = b  + learning_rate * error
```

w1 = w1 + learning_rate * error * x1_i
w2 = w2 + learning_rate * error * x2_i

Log loss is computed every 10 epochs to measure how well the model is classifying the data. Lower log loss means better classification:

log_loss = -np.mean(y * np.log(y_hat_all) + (1 - y) * np.log(1 - y_hat_all))

Dataset Visualization:

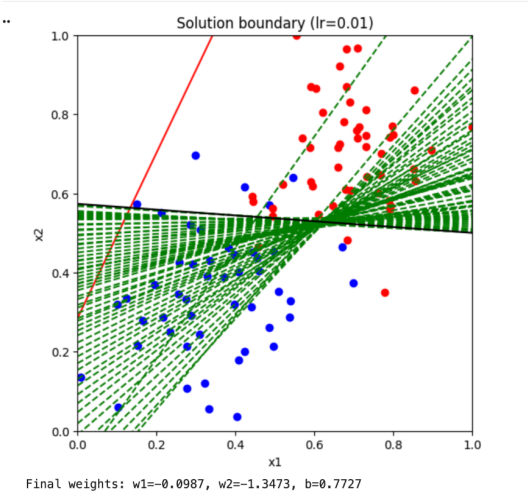


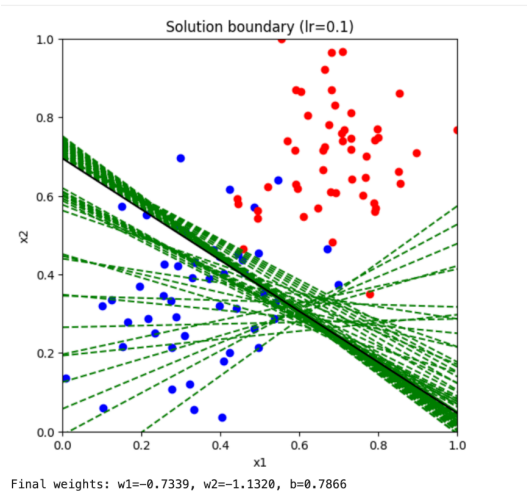
Analysis:

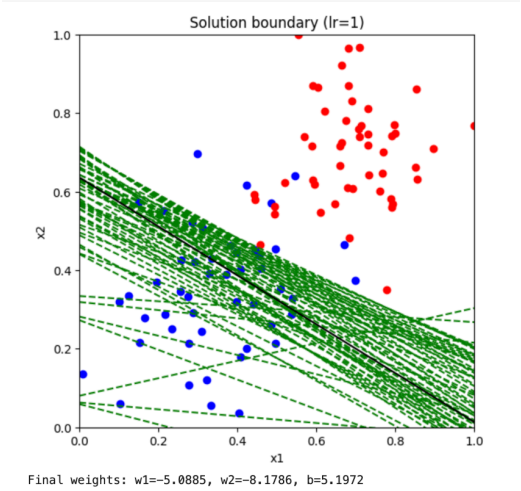
Part 1:

The final black line successfully separates the majority of red and blue points for most plots, though some overlap near the boundary is expected since the data is not perfectly linearly separable.

Effect of learning rate: A learning rate of 0.01 produced the smallest changes between iterations, with green dashed lines clustered in a tight range. A learning rate of 1 was the most chaotic, with lines covering a much wider range before converging. A learning rate of 0.1 was a middle ground between the two.





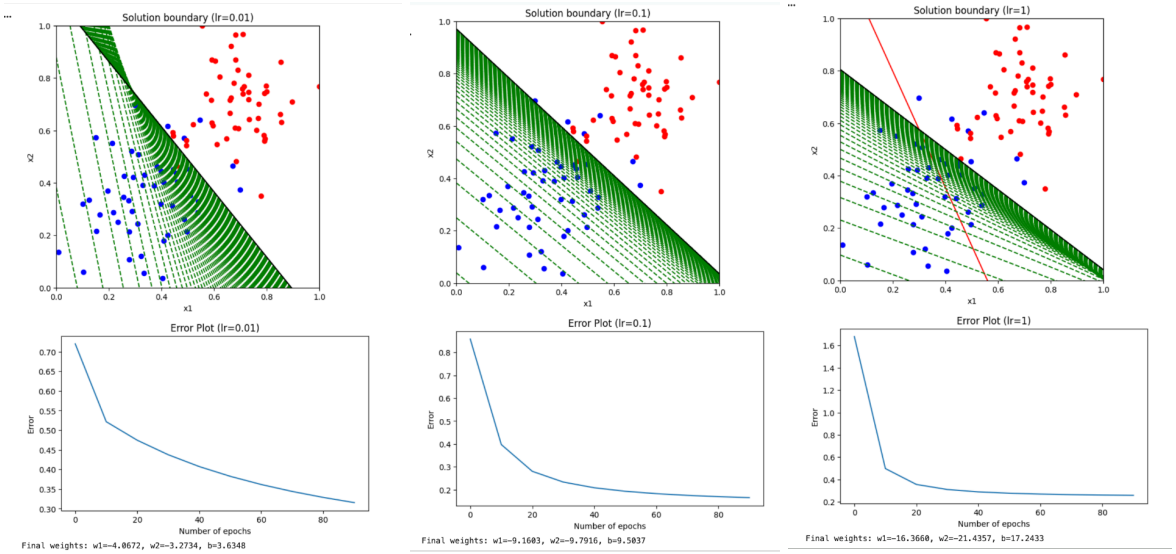


Part 2:

The gradient descent approach produced a similar final boundary to Part 1, but used a sigmoid function giving continuous probability outputs rather than binary classifications. The error plot shows log loss starting high and consistently decreasing toward a minimum, confirming the model was learning with each epoch.

Effect of learning rate: With a learning rate of 1, the boundary plot showed a large initial adjustment followed by progressively smaller adjustments as it converged. With a learning rate of 0.01 there was more variability across runs and the green lines took longer to converge, but became more precise as they approached the final line. Both learning rates showed the error dropping to a local minimum around 10 epochs before gradually reaching a lower value. The

learning rate of 1 dropped to a greater initial minimum at 10 epochs than 0.01 did.



Effect of epochs: Changing the number of epochs from 50 to 150 with a learning rate of 0.1 had less of an effect on the boundary plot compared to changing the learning rate. However increasing epochs did produce a more rounded slope in the error plot, suggesting the model had more time to gradually minimize the error.